COMP 3005

Final Project
Health and Fitness Club Management System

Individual Group Final Project 79

Mohamed Cherif Bah
101292844

2025/11/25

Introduction

The Health and Fitness Club Management System provides an all-in-one, database-driven application that focuses on the day-to-day productivity of a modern fitness club. The application has three different user roles (members, trainers, and administration) with defined access and obligations. The project uses a relational database (PostgreSQL) and an Object-Relational Mapping (ORM) framework (SQLAlchemy using Python), ensuring that the database provides data consistency, normalization, and integrity, and providing a solid support for real-life application use.

This project is intended to balance theoretical principles of conceptual database design and practical application development, demonstrating that database design can begin with theory and result in implemented applications. The solution provided by the project gives an illustration of how to combine structured data models, use case specific role-based access enforcement, and application of business rules to create a dependable and user-oriented management tool.

Project Goal

This project aims to develop and deploy a health and fitness club management system that is functional in the sense that it: (1) Allows members to manage their own profiles, track health metrics, set fitness goals, and book training or classes; (2) Provides trainers the ability to set availability, view scheduled memberships, see assigned sessions, and access relevant member progression information. (3) Gives administrative staff the ability to schedule classes and assign rooms, track equipment maintenance, and process billing. The project incorporates ORM-based data management to closely mimic real-world operations, while upholding a clean separation of roles, business constraints, and user experience.

Technology Stack & Implementation Details

Programming Language:

Python 3.10+ selected for its strong ecosystem for database-backed applications, clean syntax, and robust libraries supporting ORM, CLI development, and database migrations.

Database:

PostgreSQL provides a reliable relational database with support for advanced features such as triggers, views, indexing, and stored procedures. It ensures data integrity and transactional consistency.

ORM Framework:

SQLAlchemy ORM is used to map tables to Python classes, enabling object-oriented interaction with the database and enforcing relationships through foreign keys, cascades, and integrity rules.

Application Layer:

A Command-Line Interface (CLI) built in Python serves as the user-facing layer, supporting all workflows for Members, Trainers, and Administrative staff. This demonstrates system functionality without requiring GUI.

Database Migrations:

Alembic is used for schema versioning, automated migrations, and implementing advanced SQL features (views, triggers, stored functions).

Tools and Utilities:

pgAdmin/VsCode for database inspection, Git/GitHub for version control, dbdiagram.io for ER modeling, and standard Markdown for documentation.

Implementation Approach

The system adopts a modular architecture with separate layers for models, business logic, and presentation. SQLAlchemy ORM models define all database tables and relationships. Business logic is implemented in the services layer, covering scheduling rules, capacity checks, trainer availability, billing simulation, and maintenance workflows. The CLI simulates real system usage for all three user roles.

All database logic (except required advanced SQL features) is achieved through the ORM to satisfy the project requirement. Alembic migrations create tables, views, triggers, and stored functions.

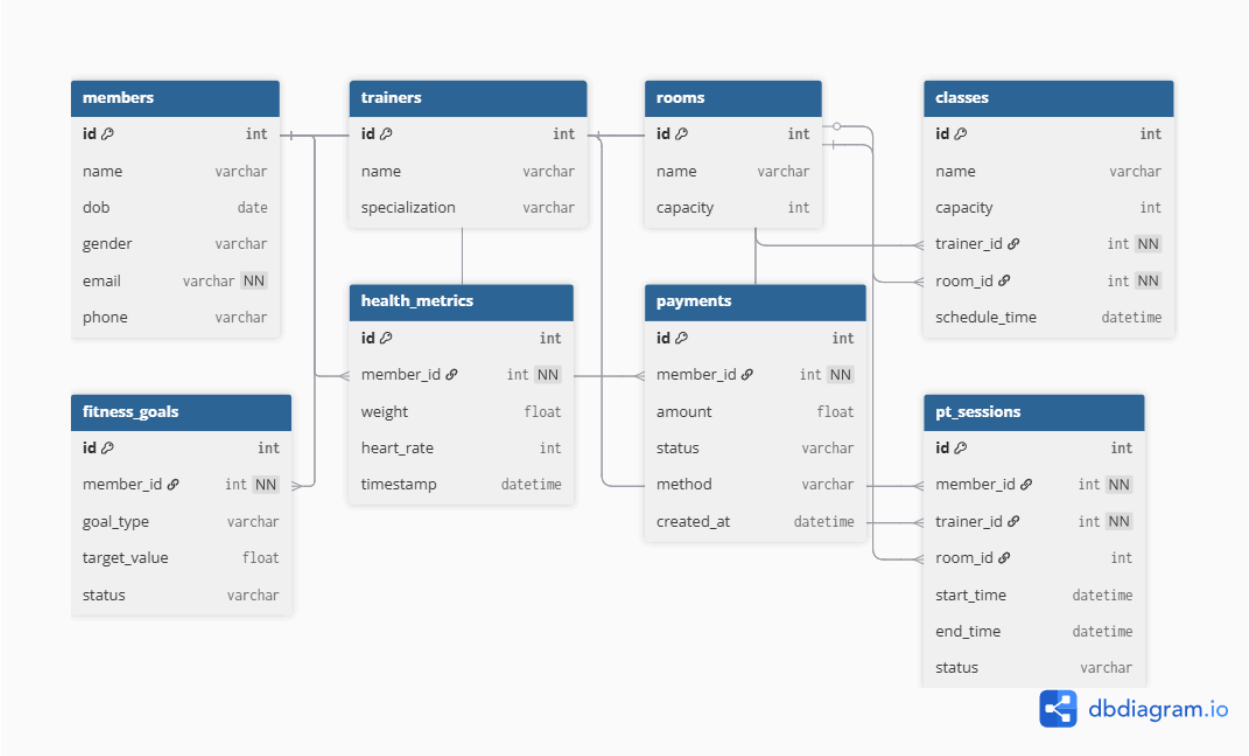
Database Design

Schema Quality and Normalization

The health and fitness club database was developed using normalization techniques to identify and eliminate redundancy and dependency issues within the early conceptual model of the

database. A well-designed normalized database conforms to the third normal form (3NF) which helps ensure that the database is as clean and efficient as possible in terms of its schema.

Initial Entity Relation diagram



1. Eliminating Repeating Groups (Addressing Issues with 1NF)

The challenge we had initially:
The first ERD draft depicted health metrics, fitness goals, and classes taken as lists within the member table (for example, a member has multiple weights and multiple fitness goals).

Implications of this scenario:
Storing more than one value per field breaks the principle of atomicity.

- New data on health will overwrite existing data for health.
- You won't be able to query historical trends.

As part of the normalizing process:
We created separate tables to hold:

- `health_metrics (one measurement per record)`
- `fitness_goals (one goal per record)`
- `class_registrations (one registration per record)`

Now that we have one record per metric, goal and registration, all rows are atomic and comply with the 1NF definition.

2. Removing Partial Dependencies (Fixing 2NF Issues)

Initial problem:
Many-to-many relationships were originally implied without bridging tables for example:

- A class having multiple members embedded in it
- A trainer having multiple availability times stored inside one row

Why it's a problem:
If a composite key is implied (e.g., member + class), attributes depending on only one side of the pair break 2NF.

Normalization fix:
We introduced bridge tables:

- `class_registrations(member_id, class_id)`
- `trainer_availabilities(id, trainer_id, start_time, end_time)`

This ensures every attribute depends on the full primary key and satisfies 2NF.

3. Removing Transitive Dependencies (Fixing 3NF Issues)

Initial problem:
The early model mixed unrelated attributes inside single tables. Examples:

- Storing trainer availability *inside* the Trainer table
- Mixing class scheduling details with room details
- Storing payment totals inside the Member table

These create transitive dependencies:

- Trainer.available_from → depends on trainer_id, but also affects class scheduling
- Member.total_payments → depends on payment rows, but indirectly on member_id

Why it's a problem:
This causes update anomalies:

- Changing availability requires editing multiple rows
- Payment correction would require recalculating stored totals manually

Normalization fix:
We separated the dependent concepts:

- Availability → `trainer_availabilities`
- Room scheduling → in `classes` and `pt_sessions`
- Payment totals → calculated dynamically using:
    - `member_payments_view`
    - `stored function get_member_total_payments`

No table contains derived or unrelated fields, the 3NF is now satisfied.

Final Database Design

Core Entities:

Member, Trainer, TrainerAvailability, HealthMetric, FitnessGoal, Class, ClassRegistration, Room, PT_Session, Payment, Equipment, EquipmentIssue.

Relationships:

Member → HealthMetric: 1-to-many

Member → FitnessGoal: 1-to-many

Member → PT_Session: 1-to-many

Member → Payment: 1-to-many

Member ↔ Class via ClassRegistration: many-to-many

Trainer → PT_Session: 1-to-many

Trainer → Class: 1-to-many

Trainer → TrainerAvailability: 1-to-many

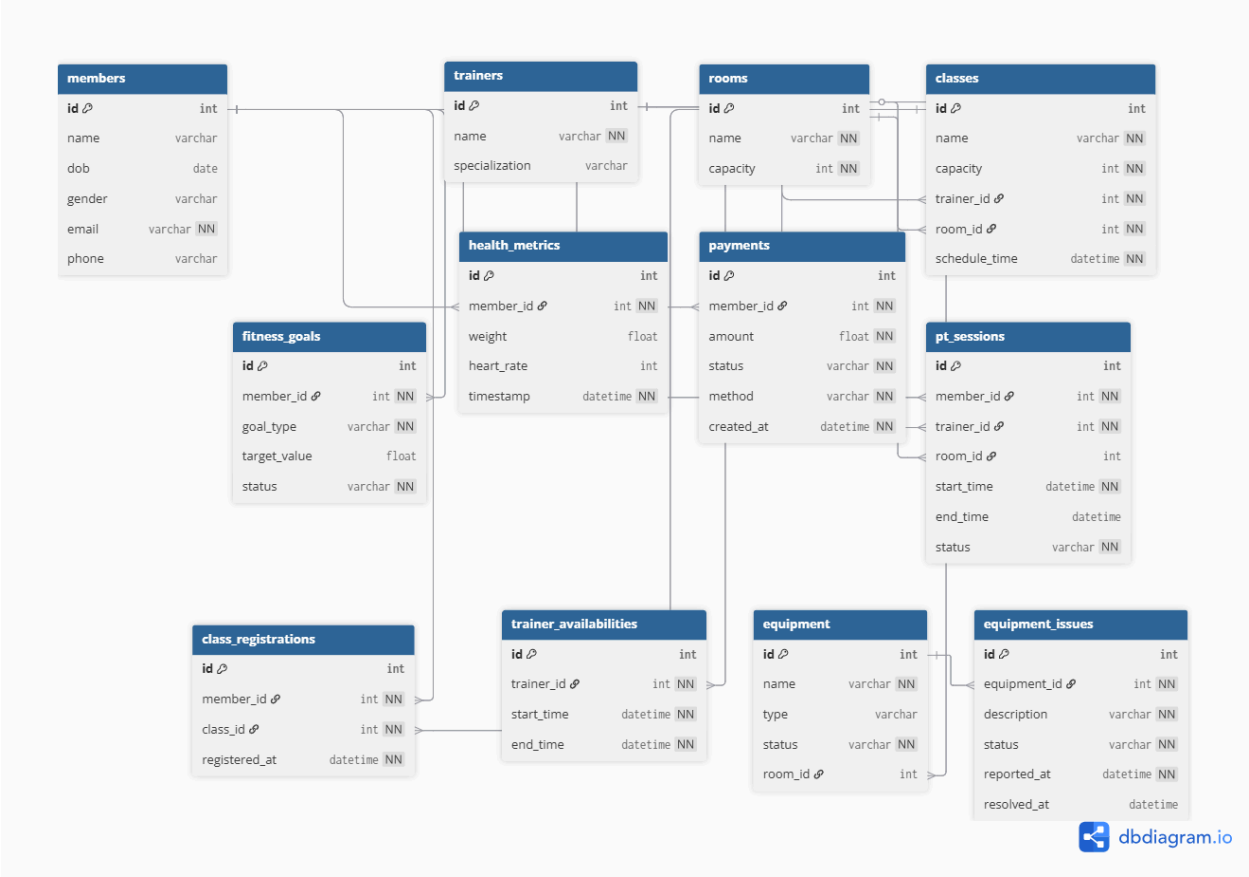
Room → Class: 1-to-many

Room → PT_Session: 1-to-many

Room → Equipment: 1-to-many

Equipment → EquipmentIssue: 1-to-many

Final Entity Relationship Diagram



ER-to-Relational Mapping

Each ER entity maps to a PostgreSQL table, with PKs and FKs enforcing relationships:

- members(id PK)
- health_metrics(id PK, member_id FK → members.id)
- fitness_goals(id PK, member_id FK → members.id)
- trainers(id PK)
- trainer_availabilities(id PK, trainer_id FK → trainers.id)
- rooms(id PK)
- equipment(id PK, room_id FK → rooms.id)
- equipment_issues(id PK, equipment_id FK → equipment.id)
- classes(id PK, trainer_id FK → trainers.id, room_id FK → rooms.id)
- class_registrations(id PK, member_id FK → members.id, class_id FK → classes.id)
- pt_sessions(id PK, member_id FK → members.id, trainer_id FK → trainers.id, room_id FK → rooms.id)
- payments(id PK, member_id FK → members.id)

Relationship Enforcement:

- All 1-to-many relationships are implemented via foreign keys.
- The many-to-many relationship between Member and Class is implemented via the ClassRegistration linking table with a UNIQUE(member_id, class_id) constraint.
- Cascading behavior on deletes is configured through ORM cascade rules.

Advanced Database Features

This system contains all necessary sophisticated database system components, including a SQL view, trigger, stored function and customized index. These components enforce business rules at the database level, improve performance, and enable automated data processing rather than requiring manual data processing.

View

The view `member_payments_view` summarizes financial information by member. It totals payments and returns a date for the most recent payment. The view has usability for reporting by allowing the system to aggregate financial data without requiring the complete join and aggregation logic every time financial data is queried.

Trigger

A trigger `trg_check_weight_goal_completion` has been implemented to automatically assess whether or not a newly inserted health metrics object met or exceeded a member's weight-loss goal. Upon triggering, the associated fitness goal is updated, indicating that the goal has been 'completed'. This design provides fidelity and guarantees updates of goals even when metrics are inserted outside of the entire application logic.

Stored Function

The stored function `get_member_total_payments(member_id)` is a database function that returns the total of all payments associated with any one member. It can be reused to compute a financial summary on the database level across reports, across views, or within manual queries that are run independently.

Index

A custom index is created on the `members.email` attribute to enable fast member lookup. Since email is frequently used for login and queries, indexing significantly improves performance. PostgreSQL also provides internal indexing for primary and foreign keys.

Application Logic & Business Rules

Business rules are enforced within the application service layer. SQLAlchemy ORM is used to interact with the relational schema, and all constraints are applied in code to ensure consistent data modifications.

Examples of enforced rules:

- Preventing double-booking of rooms, trainers, and members for sessions.
- Enforcing class capacity limits when members register.
- Restricting trainers to view only assigned member information.
- Maintaining historical data for health metrics without overwriting entries.
- Ensuring that trainer availability does not overlap.

The services layer separates domain logic from data storage, improving maintainability and clarity.

User Roles and Functional Requirements

Member

Members can register, manage profiles, track goals, log health metrics, schedule and reschedule personal training sessions, register for group classes, and view a dashboard summarizing personal data, goals, upcoming sessions, and historical progress.

Trainer

Trainers manage their availability, view their assigned PT sessions and classes, and access read-only member information (current goals and latest metrics) for members they train. Trainers cannot edit member data, maintaining role-based access integrity.

Admin

Administrative staff schedule classes, assign trainers and rooms, manage room and equipment maintenance, and handle billing. Admins can create equipment records, log equipment issues, update maintenance status, and generate payment or billing records for members.

Testing and Verification

A combination of automated service-layer tests and manual CLI testing was used to verify correctness of functionality, enforcement of business rules, and database integrity.

Automated Tests (pytest)
Automated tests were written for all major components:

Member Tests

- Registration and dashboard retrieval.
- Logging multiple health metrics and verifying that the trigger correctly marks weight-loss goals as completed.
- Class registration respecting capacity constraints.
- PT session scheduling preventing trainer, member, and room conflicts.

Trainer Tests

- Adding non-overlapping availability periods.
- Viewing assigned schedules (classes and PT sessions).
- Restricted member lookup (trainer only sees assigned members).

Admin & Database Feature Tests

- Creating equipment, logging issues, updating maintenance status.
- Creating and updating payments.
- Validating correctness of:
- member_payments_view (SQL view)
- get_member_total_payments (stored function)

All automated tests confirmed correct behavior of business logic and database constraints.

Manual CLI Testing

Manual testing focused on verifying complete workflows:

Member

- Registration/login
- Dashboard (latest metrics, goals, upcoming sessions)
- Goal creation, health metric logging
- Class registration/cancellation
- PT session scheduling/rescheduling
- Payment viewing

Trainer

- Availability management
- Schedule viewing
- PT session status updates
- Read-only member lookup

Admin

- Room and class management

- Equipment and maintenance tracking
- Payment creation and updates
- Verification Summary

Testing confirmed that:

All workflows for Member, Trainer, and Admin roles operate correctly.

Business rules (capacity, double-booking prevention, non-overlapping availability, restricted access) are enforced.

Advanced database features (view, trigger, stored function, index) behave as intended.

The system maintains data integrity, historical records, and normalization throughout.

Conclusion

The Health and Fitness Club Management System showcases a complete implementation of normalized relational database with full ORM integration, advanced database features, and application-layer business rules. It enforces strict scheduling and capacity rules, supports role-based access, and provides accurate historical and real-time data. The modular architecture allows for maintainability and future expandability, including possibly adding a graphical interface or REST API.